

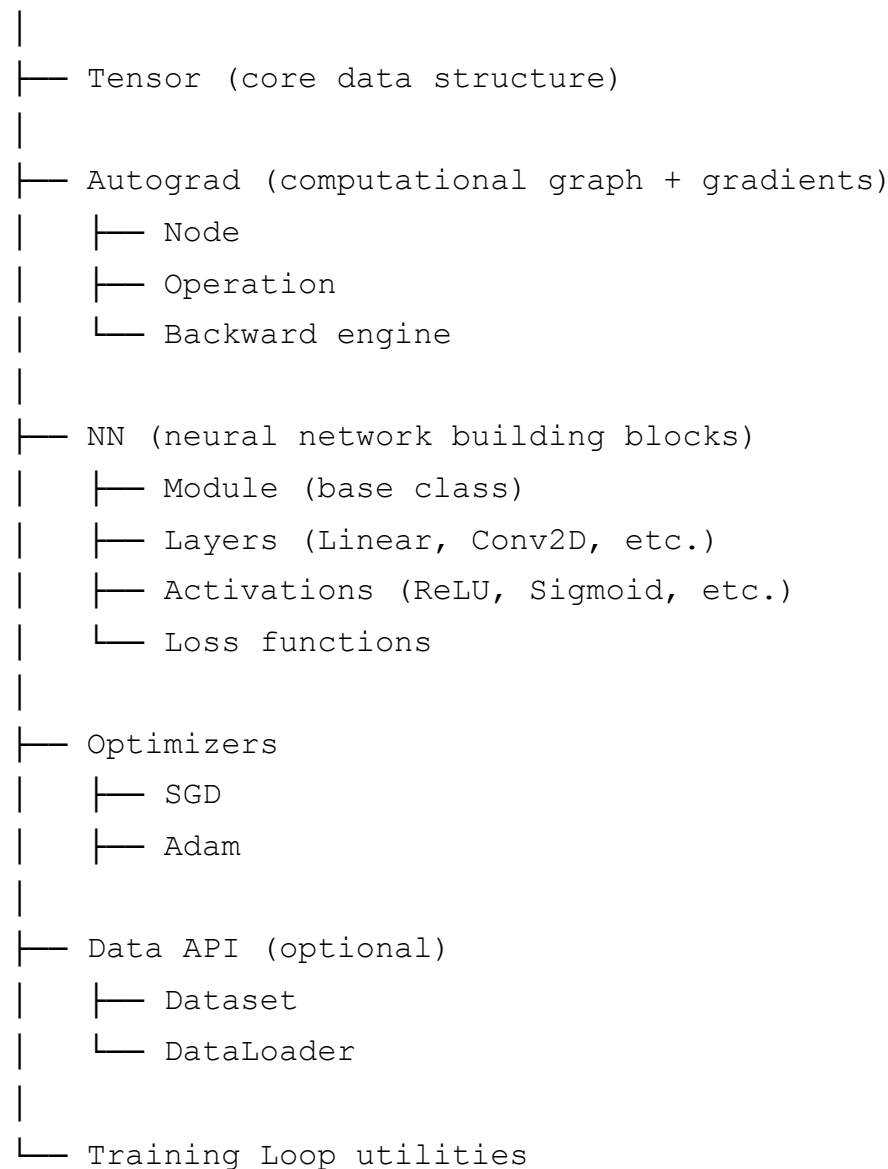
If you're building a **neural-network library from scratch**, the internal **object tree** (the structure of classes and modules) should be clean, modular, and similar to PyTorch or Keras—but simpler.

Below is a clear, practical guide to how the **architecture/tree of an NN library** should look.

Neural-Network Library Architecture (from scratch)

A good NN library has these layers of abstraction:

Library



Below I explain each part and show what the class tree looks like.

1 Tensor (core class)

This is the most important object.

Tensor

- └ data (numpy array)
- └ grad (same shape)
- └ requires_grad (bool)
- └ grad_fn (pointer to op that created it)

2 Autograd system

Node / Op Tree

Each operation builds a node in the computation graph:

Op

- └ inputs -> [Tensor, Tensor, ...]
- └ forward(...)
- └ backward(grad_output)

Tensors created by operations hold a reference:

Tensor.grad_fn → Op instance

Example:

`c = a + b`

`c.grad_fn` is `AddOperation`

3 Module (base neural network class)

Similar to PyTorch's `nn.Module`.

Module

- └ parameters() → list of tensors
- └ children() → submodules

- └─ forward(...)
- └─ __call__ → wraps forward + hooks

4 Layers

Organized under `nn/` :

`Linear (Module)`

- └─ weight
- └─ bias

`ReLU (Module)`

- └─ no params

`Sequential (Module)`

- └─ list of submodules

`Conv2D (Module)`

- └─ weight
- └─ bias

Your folder tree would look like:

```
nn/  
└─ module.py  
└─ linear.py  
└─ activations.py  
└─ conv.py
```

5 Loss functions

Losses also inherit from module:

`Loss (Module)`

- └─ forward(pred, target)
- └─ backward (handled by autograd)

Examples:

MSELoss

CrossEntropyLoss

6 Optimizers

Optimizers operate on parameters of module.

Optimizer

- └─ params (list of tensors)
- └─ step()

SGD(Optimizer)

- └─ momentum buffer

Adam(Optimizer)

- └─ m (1st moment)
- └─ v (2nd moment)

7 Training Utilities

Optional but helpful:

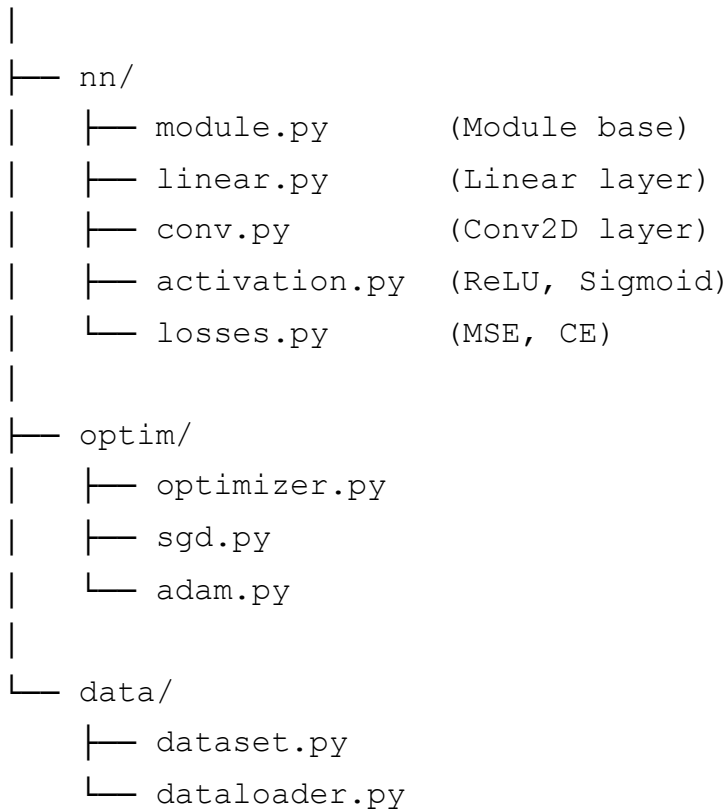
Trainer

- └─ train_step
- └─ fit

Full Class Tree (simple version)

mytorch/

- |
- └─ tensor.py
 - └─ Tensor
- |
- └─ autograd/
 - └─ node.py (Op base class)
 - └─ engine.py (backprop engine)
 - └─ ops.py (Add, MatMul, etc.)

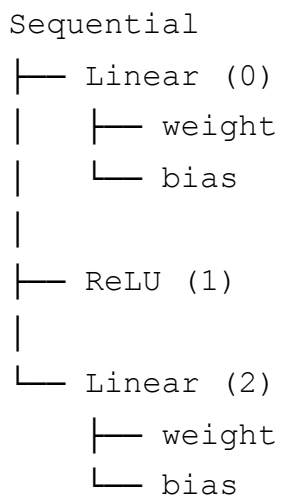


Example: Modules and parameters tree

If you build a simple model:

```
model = Sequential(
    Linear(10, 20),
    ReLU(),
    Linear(20, 1)
)
```

Internal tree:



Want code examples?

I can generate:

- ✓ full minimal NN library (50–80 lines)
- ✓ full autograd engine
- ✓ PyTorch-like module system
- ✓ optimizer + training loop
- ✓ folder structure templates

Just tell me what level of detail and whether you want **Python code**, **UML diagrams**, or both.